Capstone Project Architecture: Hermes-Clawdbot

Autonomous Multi-Platform Agent Engine with MicroVM Sandboxing, Durable Workflows & AST Context Compaction

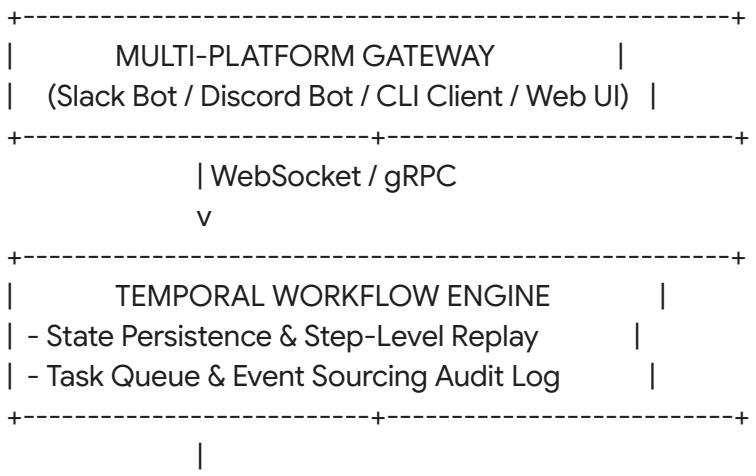
Executive Summary

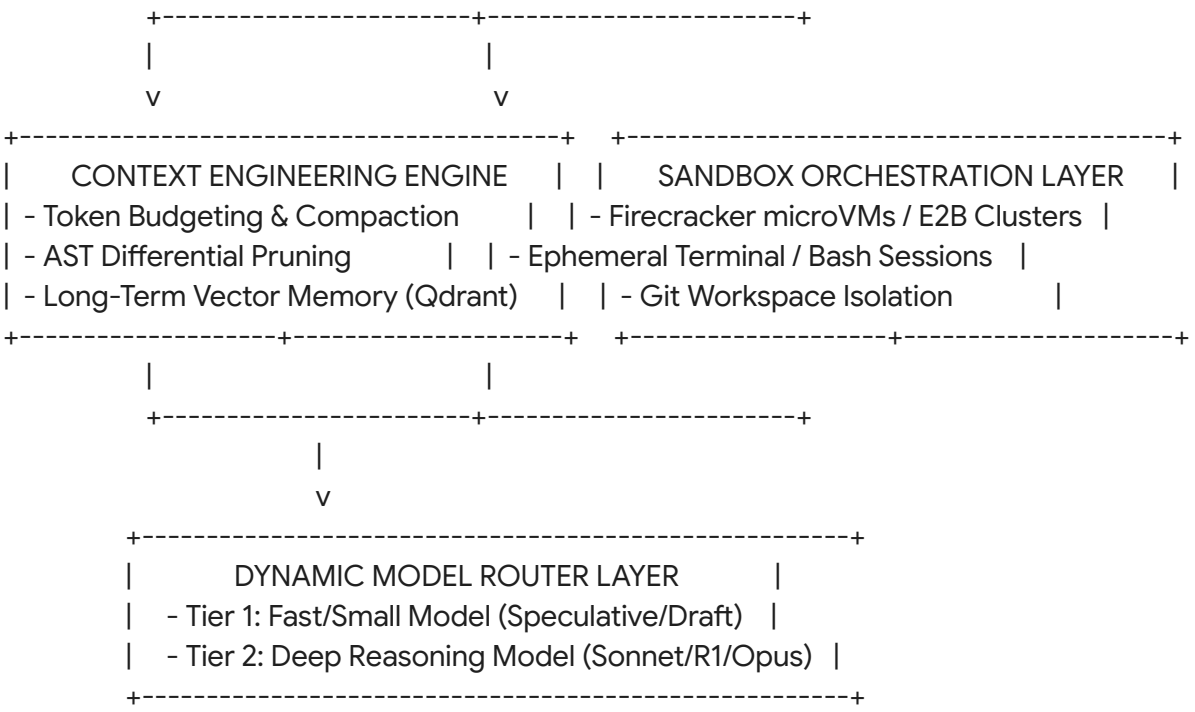
Hermes-Clawdbot transforms standard chatbot/agent implementations into an enterprise-grade autonomous execution engine. Taking inspiration from state-of-the-art developer agents (such as Devin, Claude Code, and Manis), this system executes complex, long-running software engineering and system tasks across multiple user platforms (Slack, Discord, CLI, and Web Dashboard).

It specifically addresses the critical real-world bottlenecks highlighted by industry engineers:

1. **Secure Execution:** Running untrusted code in ephemeral, isolated Firecracker microVM sandboxes.
2. **Durability:** Surviving server crashes and network drops over multi-hour agent trajectories using Temporal workflow state machinery.
3. **Context Engineering:** Preventing context degradation and token bloat across $500k+$ token conversations using AST-aware compaction and observation pruning.
4. **Evaluation:** Benchmarking agent capabilities against real repository issues with custom execution evals.

1. System Architecture & High-Level Topology





2. Core Technical Pillars & Modules

Module A: Ephemeral MicroVM Sandboxing (Firecracker / E2B)

- **Problem:** Executing shell commands, code modifications, or package installs on host machines poses high security risks and clean-up challenges.
- **Solution:** Every task spawns an isolated microVM sandbox with an independent filesystem, Linux kernel, and non-root privileges.
- **Implementation:**
  - Build a custom agent worker that provisions microVM sandboxes via Firecracker or the E2B SDK in under 500ms.
  - Mount repository state, establish WebSocket streaming for shell output (stdout/stderr), and isolate file operations.
  - Automatically destroy environments upon task completion or freeze/snapshot state for post-mortem debugging.

Module B: Durable Workflow Execution (Temporal.io Integration)

- **Problem:** Long-running agent tasks (15–60 minutes) fail completely if the orchestration server restarts or drops network connectivity.
- **Solution:** Reconstruct the agent loop as a **Durable Temporal Workflow**.
- **Implementation:**
  - Model every agent iteration (Thought -> Action -> Tool Call -> Observation) as a

deterministic Temporal Activity.
    - Record full state histories in Temporal event logs. If a node crashes midway through step 14 of an issue fix, the system recovers, rehydrates state, reconnects to the active Firecracker sandbox, and resumes step 15 seamlessly.

Module C: Context Engineering & Anti-Rot Engine

- **Problem:** Extended terminal logs and large codebase reads trigger context rot, hallucinations, and high API costs.
- **Solution:** Implement a hierarchical context compaction pipeline (inspired by Manis & Claude Code).
- **Implementation:**
    - **Observation Truncation:** Pass terminal outputs longer than 50 lines through AST-guided or semantic line summarization, preserving error stack traces and trimming noise.
    - **Context Compactor:** When context reaches 75% capacity, generate a structured summary of previous attempts, active file structures, and key discoveries while pruning outdated tool outputs.
    - **File Diff Memory:** Maintain long-term code changes as clean Git patches rather than repeatedly feeding entire modified files.

Module D: Dynamic Model Routing & Cost Policy Layer

- **Problem:** Route tasks efficiently to avoid overspending on high-parameter reasoning models for basic commands.
- **Solution:** A model router that assigns sub-tasks dynamically.
- **Implementation:**
    - Route simple file reads, bash command generations, and formatting tasks to lightweight models (e.g., Llama-3-8B / Claude Haiku).
    - Escalate complex bug diagnosis, code refactoring, and strategy formulation to high-capability models (e.g., Claude 3.5 Sonnet / DeepSeek-R1).
    - Enforce organization-level credit limits, safety filters, and personal prompt blocking.

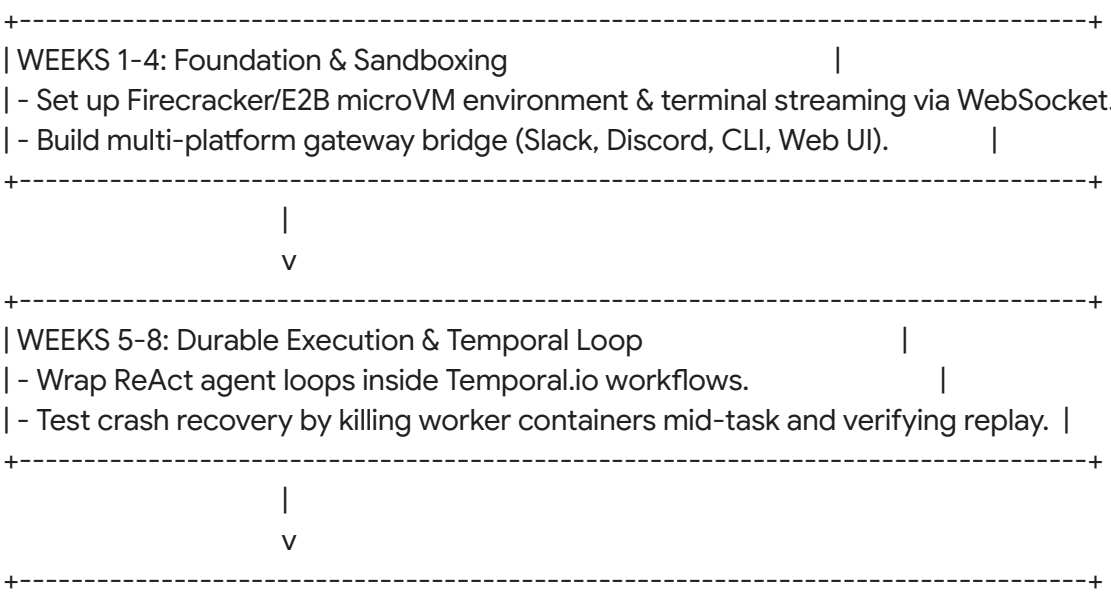
Module E: Custom Repository Evaluation Engine (SWE-Bench Style)

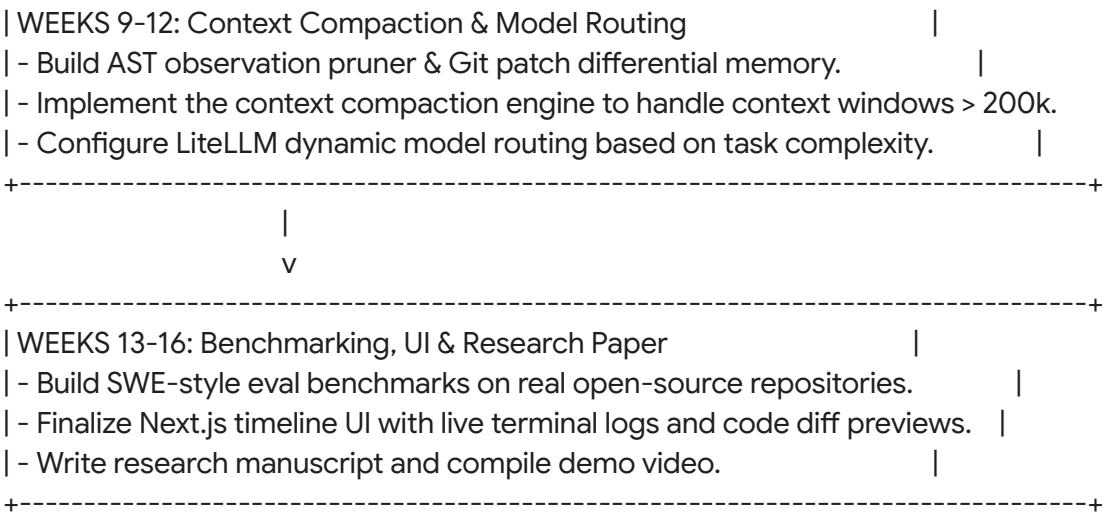
- **Problem:** Standard LLM metrics fail to accurately evaluate real-world agent reliability.
- **Solution:** An automated benchmarking platform to test agent performance on specific codebases (e.g., test suite execution on target repositories like Dub.sh or open-source projects).
- **Implementation:**
    - Create standard test suites: Given a repository issue description, execute the agent end-to-end, run unit tests inside the sandbox, and verify pass/fail rates.
    - Track metrics: Task completion rate, total token expenditure, tool-call accuracy, and resolution time.

3. Recommended Tech Stack

Layer	Recommended Technologies
Agent Core & Orchestration	Node.js/TypeScript (or Python), Temporal.io SDK, LangGraph / Custom ReAct Loop
Sandboxing & Isolation	Firecracker MicroVMs, E2B SDK, Docker Sandboxes
Frontend & Control Center	Next.js 15, Tailwind CSS, Xterm.js (for live terminal streaming), WebSockets
Multi-Platform Adapters	@slack/bolt, discord.js, Custom CLI Node Package
Database & Vector Memory	PostgreSQL (State/Users), Qdrant / Milvus (Semantic Code & Memory Search)
LLM Gateway & Router	LiteLLM Proxy / OpenRouter API, Anthropic SDK, vLLM
Evals & Monitoring	Custom Dockerized SWE-bench runner, LangSmith / Phoenix Tracing

4. 16-Week Capstone Roadmap





5. Resume Highlights & Research Paper Angle

Resume bullet points:

- *Engineered an autonomous multi-platform agent platform using Temporal.io for durable workflow state execution, enabling 60+ minute fault-tolerant task execution across Slack, Discord, and Web.*
- *Architected a microVM sandbox orchestration layer (Firecracker/E2B) executing arbitrary shell commands and code modifications in sub-second ephemeral environments.*
- *Developed an AST-guided context compaction pipeline reducing token consumption by 45% on multi-step software engineering trajectories.*
- *Built a custom SWE-bench testing framework to benchmark model tool-calling accuracy and issue resolution across open-source repositories.*

Research Paper Angle:

Title: *"Durable Multi-Platform Agent Execution with MicroVM Sandboxing and AST-Guided Context Compaction"*

- **Key Focus:** Benchmarking context rot mitigation strategies and crash-recovery performance in multi-turn software development agents operating over extended context windows.